GENERAL SIR JOHN KOTELAWALA DEFENCE UNIVERSITY



BACHELOR OF SCIENCE (HONORS) INFORMATION TECHNOLOGY
INTAKE 41
SOFTWARE QUALITY ASSURANCE
(IT3153)

Assignment 05

Automated UI Test Suite for LinkedIn

Name: WAMD Wijethunga
Reg.No: D/BIT/24/0017
Field: IT

Sub Date: 07/11/2026

Table of Contents

1. Introduction 2

2. Test design 2

3. Test Cases 3

4. Screenshots 4

5. Locators and why 6

6. Challenges and how I addressed them 7

7. Reflection 7

1. Introduction

This report describes an automated UI test suite for LinkedIn. It is written in Java 17 with Selenium WebDriver 4.25.0 and TestNG 7.10.2, built with Maven, and run in Chrome from VS Code. The suite automates four test cases: a valid login, a login with a wrong password, a keyword search, and opening the first search result. The code follows the Page Object Model, so the locators and page actions live in page classes and the test classes only call them and assert the outcome. All runs used my own LinkedIn account, as the assignment requires.

2. Test design

- **Base test class.** BaseTest will open a maximised Chrome browser at the start of each test. A screenshot will be taken with the filename based upon the test name and result. This will be saved into the 'screenshots' folder. The final page will then remain open for 12 seconds (can be set in result.pause.seconds) so that the results can be seen in the screen recording.
- **Page classes.** BasePage holds the wait, click and type helpers. LoginPage opens the login page, signs in, reads the error message and handles security checkpoints. FeedPage confirms that the feed loaded and runs the global search. SearchResultsPage opens the People results and clicks a result. DetailPage reads the heading of the page that was opened. The test classes are LoginTest, SearchTest and SearchResultTest.
- **Waits.** Implicit waits specify a global timeout for all element lookups. Explicit waits (WebDriverWait) can be used to wait for 1 condition. Fluent waits are explicit waits that allow you to specify a polling time. None of the implicit waits have been used. WebDriverWait timeouts have been used for all page classes (20s). However, a helper method has been created to wait for the first displayed element. This is required due to having duplicate elements on LinkedIn pages. SearchResultsPage has used a FluentWait (20s, 500ms polling, ignores NoSuchElement and StaleElement exceptions). The driver has a page load timeout of 90s. The only use of Thread.sleep is for the 12 second pause whilst viewing the page and for the 2 second red border around the selected result. These are purely for display purposes and are not used for synchronisation.
- **Credentials and test data.** The e-mail and password are read from config.properties, which is listed in .gitignore and so is never committed; the environment variables LINKEDIN_USERNAME and LINKEDIN_PASSWORD can override them. The two search terms are in the same file: “system testing” for the search test and “Satya Nadella” for the open-result test.
- **Security checks.** If LinkedIn shows a CAPTCHA or checkpoint after sign-in, LoginPage detects it from the URL (checkpoint or challenge) and waits up to 3 minutes for it to be completed by hand in the browser. The suite never tries to bypass it.

3. Test Cases

ID	Description	Steps	Expected Result	Actual Result	Pass / Fail
TC01	Valid login	Open /login; enter the account e-mail and password; click Sign in	URL contains /feed and the navigation bar is visible	The home feed opened and the navigation bar was displayed	Pass
TC02	Login with an invalid password	Open /login; enter the account e-mail and a wrong password; click Sign in	An error message is displayed	“Wrong email or password.” appeared under the password field	Pass
TC03	Search for a topic	Log in; click the global search box; type “system testing”; press Enter	URL contains /search/results and at least one result is visible	The results page opened (keywords=system+testing) and results were listed	Pass
TC04	Open the first search result	Log in; search “Satya Nadella”; open the People results; click the first result	URL contains the path of the clicked result and the page heading contains its name	Satya Nadella’s profile opened and its heading matched the result that was clicked	Pass

4. Screenshots

The browser screenshots below were captured automatically by the code (ScreenshotUtil) at the end of each test. Figure 1 is cropped at the bottom and the e-mail address in Figure 3 is hidden, to keep personal information out of the report.

Valid login

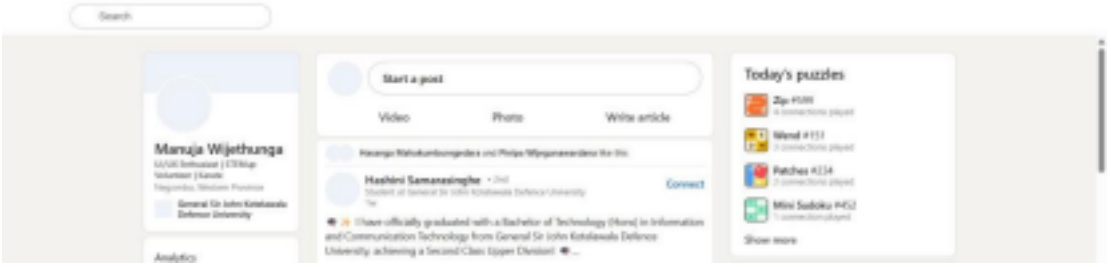


Figure 1: Home feed shown after a successful login (TC01)

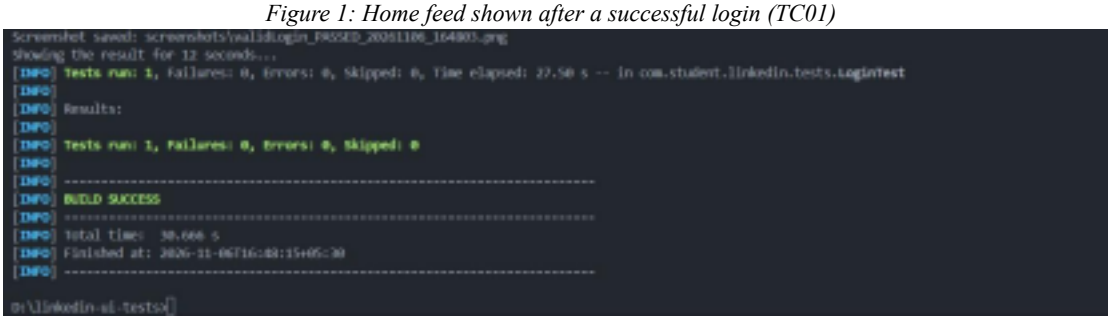


Figure 2: Console output of the TC01 run (Tests run: 1, Failures: 0, BUILD SUCCESS)

Invalid login

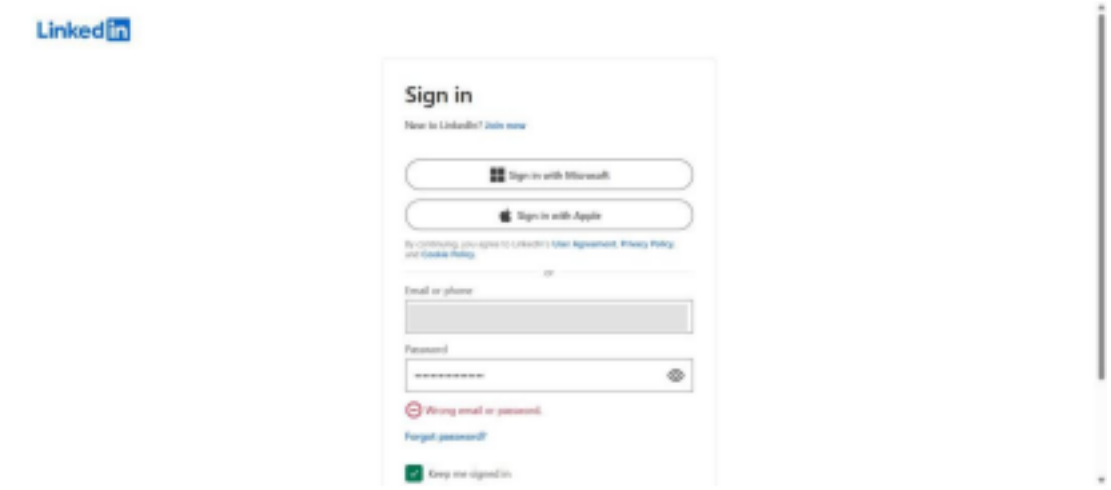


Figure 3: LinkedIn's "Wrong email or password." message after the invalid login (TC02)

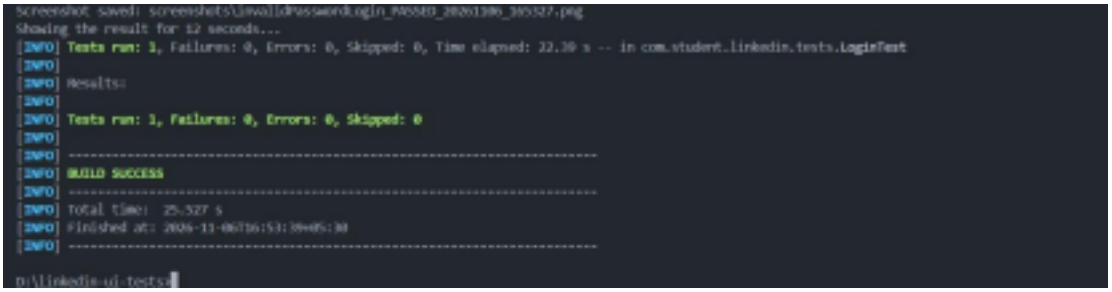


Figure 4: Console output of the TC02 run

Search test

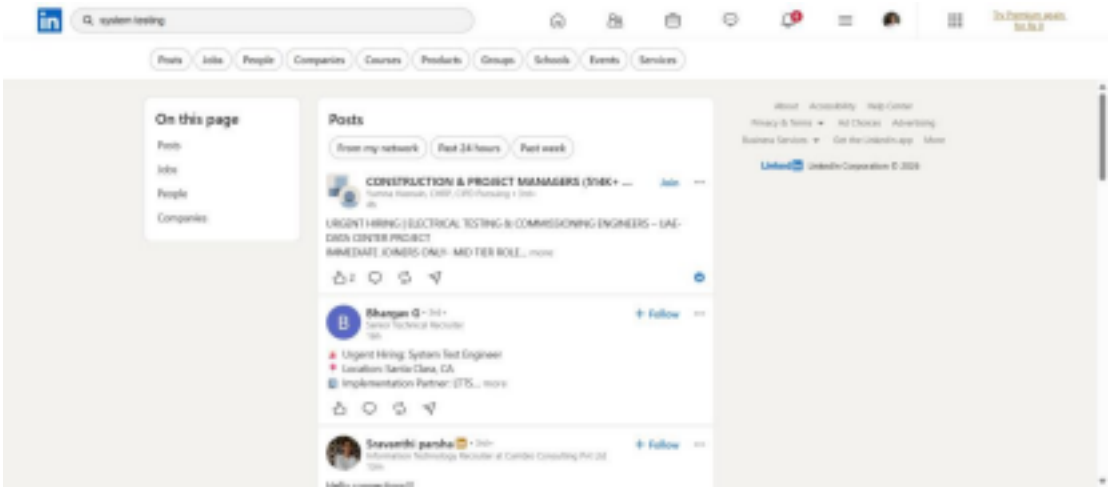


Figure 5: Search results page for "system testing" (TC03)

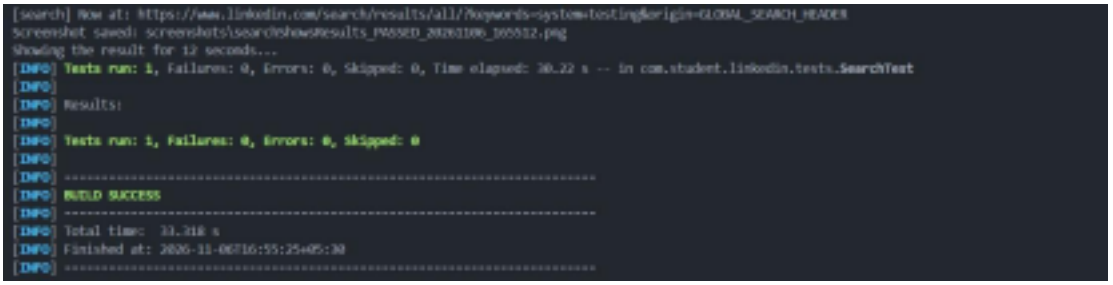


Figure 6: Console output of the TC03 run, showing the results URL

Open result

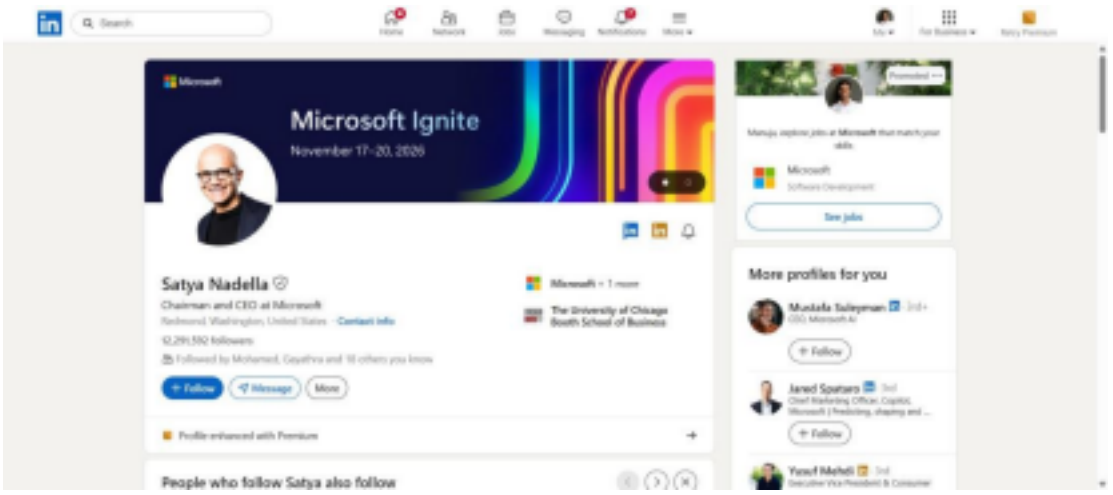


Figure 7: Profile page opened by clicking the first search result for "Satya Nadella" (TC04)

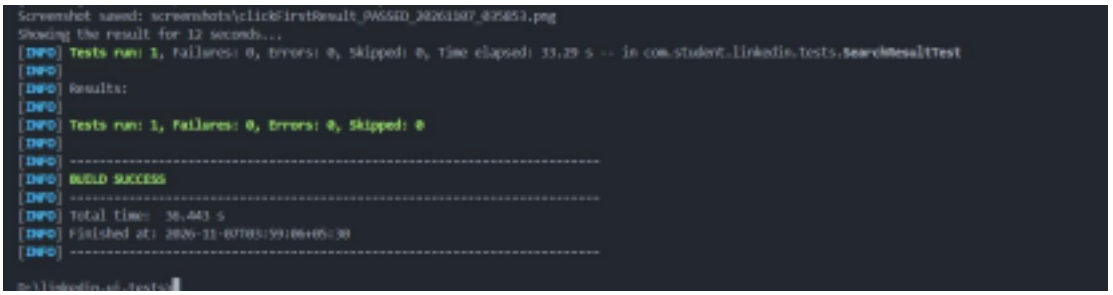


Figure 8: Console output of the TC04 run

5. Locators and why

Page	Element	Locator in code	Why I chose it
Login	E-mail box	`input[autocomplete*='username'], input[type='email']`	The ids and class names are generated and change, so I used the stable type and autocomplete attributes. The page also has hidden duplicate fields, so the helper takes the first displayed match.
Login	Password box	`input[type='password']`	Same reason as the e-mail box; the type attribute is stable.
Login	Sign in button	`//button[normalize-space()='Sign in']`	The page has several buttons (Google, Microsoft, Apple), so the visible text identifies the right one.
Login	Error message	`//*[@role='alert']` or an element whose text contains "Wrong email or password" (and similar wordings)	The message has no stable id or class; its role and its text are the stable parts.
Feed	Login success	URL contains /feed, and nav, header, [role='navigation'], [role='banner'] is displayed	The URL is a reliable signal. A structural selector avoids LinkedIn's generated class names.
Feed	Search box	`input[placeholder*='Search' i], input[aria label*='Search' i], input[role='combobox'], input[type='search']`	Several attribute alternatives, so the locator still works if one attribute changes.
Results	Results page	URL contains /search/results	A URL pattern changes less often than the page layout.
Results	People results	/search/results/people/?keywords=<name> (built by the code)	A search opens on the Posts tab, whose links belong to post authors. The People results list people.
Results	Result links	`//a[contains(@href,'/in/') or contains(@href,'/company/') or contains(@href,'/school/')][not(ancestor::nav)][not(ancestor::header)]`, displayed links only	Based on where the link points, not on styling. Leaving out nav and header avoids the link to my own profile in the top bar.
Opened page	Heading	h1, h2 or h3 outside nav and header, whose text contains the clicked name	A plain h1 first matched "0 notifications" in the top bar. Limiting it to the page body and requiring the name makes the check meaningful. The tab title is the fallback.

The locators that had to be changed during the project were the login ids (#username and #password), and my first guesses for the navigation bar, the error message and the page heading.

6. Challenges and how I addressed them

- **Page load timeout.** The first runs failed with "Timed out receiving message from renderer" while the login page was loading. I set the page load strategy to EAGER (return when the page structure is ready) and a 90 s page-load timeout, and left the waiting for individual elements to explicit waits.
- **A login page that had changed.** The locators I started with (#username and name=session_key) matched nothing, and the ids shown in DevTools were generated. I listed every input on the page from the console, chose the type and autocomplete attributes, and filtered for displayed elements because the page has hidden duplicate fields.
- **Stale elements.** LinkedIn redraws its top bar just after login, so the search box and the navigation bar sometimes went stale between being found and being used (StaleElementReferenceException). The helpers now look the element up again and retry, and the fluent wait ignores this exception.
- **Screenshot taken too late.** My first screenshot code ran after the browser had closed and crashed the run with "invalid session id". The screenshot is now taken in the teardown method before driver.quit().
- **The wrong result was clicked.** A search opens on the Posts tab, so the first /in/ link belonged to the author of a post and not to the person searched for. The test now opens the People results, picks the first result that contains the searched name, outlines it in red, and checks that both the URL and the page heading match the result that was clicked. The click is tried as a mouse click, a normal click and a JavaScript click in turn, and the test fails if the page had to be opened by URL instead.
- **CAPTCHA and security checks.** LinkedIn can show a verification step during automated logins. The suite waits for it to be completed by hand and never tries to bypass it, and the credentials are kept out of the code and out of Git.

7. Reflection

A UI test suite that drives a site I do not control is fragile. LinkedIn changes its page structure without notice: the ids on the login page are generated, class names change, and a layout I inspected on one day looked different on the next, so locators broke even though my code had not changed. Its bot detection, and terms that do not allow automated access, mean a run can be interrupted by a security check or put an account at risk, so this suite suits learning with my own account and a small number of runs, not continuous testing.

I also do not control the data. The search results, and so the first result, depend on the account and on the time, which is why the tests assert on URL patterns and on the name of the result that was clicked instead of on fixed text. And when a UI test fails on a site I cannot inspect, the failure says little about the cause. For real projects I would test an application I own in a staging environment, with API tests covering the logic and a small number of UI tests on top.

GIT Repo: https://github.com/Manuja-Wijethunga/linkedin-ui-tests.git